

TP5 : Le MUD en Dart

Qualité de développement — premiers pas en Dart

BTS2 - SLAM

2026-2027

Vous connaissez déjà les boîtes et les choses du MUD : vous les avez développées en Java, en TDD, au TP4. Dans ce TP, vous refaites le même travail **en Dart**, le langage de Flutter. Le problème est connu : vous pouvez vous concentrer sur le nouveau langage.

Objectifs

À la fin de ce TP, vous saurez :

- créer, lancer et tester un projet Dart en ligne de commande ;
- écrire une classe Dart : attributs, constructeur, méthodes, getters ;
- utiliser les **paramètres nommés**, qui remplacent la surcharge de Java ;
- utiliser la **null safety** : la différence entre `int` et `int?` ;
- écrire des tests avec `package: test`, sans abandonner la démarche TDD et les branches git du TP4.

Rappel

Supports de cours à garder ouverts pendant le TP :



- *Dart 1 — Premiers pas ;*
- *Dart 2 — Classes et objets ;*
- *Dart 3 — Collections et tests.*

Récupérez le projet

Comme au TP4, partez du dépôt modèle sur GitHub :

<https://github.com/roza/mud-dart-tp5>

Bouton vert **Use this template**, puis **Create a new repository**. Nommez votre dépôt `mud-dart-tp5`, cochez **Private**, validez, puis ajoutez-moi en collaborateur (compte roza).

Clonez votre dépôt (remplacez VOTRE-COMPTE par votre identifiant GitHub) :

```
$ git clone https://github.com/VOTRE-COMPTE/mud-dart-tp5.git
$ cd mud-dart-tp5
$ ls
LISEZMOI.md  TP5.pdf  analysis_options.yaml  bin  lib  pubspec.yaml  test
```

Attention



Comme au TP4 : **pas** de `git init` en plus du `git clone`.

Premier contact avec l'outil dart

Vérifiez d'abord que Dart est installé :

```
$ dart --version
Dart SDK version: 3.11.4 (stable) ...
```

Téléchargez ensuite les dépendances du projet (la bibliothèque de tests et les règles de style) :

```
$ dart pub get
```

Puis lancez le programme principal :

```
$ dart run
Bienvenue dans le MUD en Dart!
```

À retenir

Tout passe par une seule commande, dart :

Commande	Rôle	Équivalent Java du TP4
dart pub get	télécharge les dépendances	copier le .jar de JUnit
dart run	lance bin/mud.dart	javac puis java
dart test	lance tous les tests du dossier test/	java -jar \$J execute
		...
 dart analyze	signale erreurs et mauvaises pratiques	(rien d'équivalent)

Plus de compilation séparée, plus de *classpath* ni de variable J.

Organisation du projet

- pubspec.yaml : la carte d'identité du projet (nom : mud) et ses dépendances. Fini les .jar copiés à la main dans le dossier, comme aux TP3 et TP4 : dart pub get télécharge ce qui est déclaré ici.
- lib/ : le code de votre bibliothèque (box.dart, thing.dart) ;
- test/ : les tests, un fichier xxx_test.dart par classe ;
- bin/mud.dart : le programme principal, avec sa fonction main().

Question 1

Ouvrez pubspec.yaml. Quelles sont les deux dépendances du projet ? Pourquoi sont-elles rangées dans dev_dependencies et non dans dependencies ?

Astuce



Dans VS Code, installez l'extension **Dart**. Elle affiche les erreurs pendant la frappe, complète le code, et ajoute un lien *Run* au-dessus de chaque test.

Rappel de la méthodologie

La méthode est **exactement** celle du TP4. Seules les commandes changent.

À retenir

Pour chaque élément du backlog :

1. depuis main à jour, créer une branche `test-<feature>` ;
2. traduire l'élément en **un test** ;
3. lancer `dart test` : le test doit échouer **par manque de fonctionnalité** ;
4. commiter le test ;
5. implémenter la fonctionnalité ;
6. lancer `dart test` : tous les tests doivent passer ;
7. commiter, pousser, créer la pull request, fusionner ;
8. revenir sur main, `git pull`, supprimer la branche locale.



Le fichier `.gitignore` est déjà fourni : il exclut le dossier `.dart_tool/`, que Dart génère lui-même.

Premiers éléments du backlog : ensemble

On veut pouvoir créer des boîtes

Question 2

Traduisez cet item de backlog en un test.

On crée le fichier `test/box_test.dart` :

```
import 'package:mud/box.dart';
import 'package:test/test.dart';

void main() {
  test('on peut créer une boîte', () {
    final b = Box();
    expect(b, isA<Box>());
  });
}
```

Quelques remarques, par rapport à JUnit :

- un fichier de test est un programme normal, avec sa fonction `main()` ;
- chaque test est un appel à la fonction `test(description, fonction)`. La description est une phrase en français : elle s'affiche quand le test échoue ;
- `() { ... }` est une **fonction anonyme**, comme la lambda `() -> { ... }` de `assertThrows` au TP4 ;
- `expect(valeur, matcher)` remplace `assertEquals` : on écrit d'abord la valeur obtenue, puis ce qu'on attend ;
- `final b = Box();` : pas de `new`, et pas besoin d'écrire le type : Dart le **déduit** (inférence de type). `final` veut dire que `b` ne sera pas réaffectée.

`package:mud/box.dart` désigne le fichier `lib/box.dart` du projet `mud` (le nom qui figure dans `pubspec.yaml`).

On lance les tests :

```
$ dart test
00:00 +0 -1: loading test/box_test.dart [E]
```

```
Failed to load "test/box_test.dart":
test/box_test.dart:1:8: Error: Error when reading 'lib/box.dart':
No such file or directory
...
test/box_test.dart:7:19: Error: 'Box' isn't a type.
```

Le test échoue, et c'est normal : Box n'existe pas encore. On commite le test dans une branche :

```
$ git switch -c test-box-create
$ git add test/box_test.dart
$ git commit -m "Biblio Box: ajout test création"
```

Astuce



git switch -c <branche> crée la branche **et** bascule dessus : c'est le raccourci de git branch <branche> suivi de git switch <branche>.

On implémente ensuite la fonctionnalité dans lib/box.dart :

```
class Box {}
```

Une classe vide suffit : Dart fournit un constructeur par défaut.

```
$ dart test
00:00 +1: All tests passed!
```

Commitez, poussez, faites la pull request et fusionnez, comme au TP4.

On veut pouvoir mettre des choses dans une boîte

Au TP4, on avait d'abord mis des String dans la boîte, avant de passer aux Thing. On va directement aux Thing.

Question 3

Écrivez le test on peut mettre des choses dedans, dans une nouvelle branche test-box-add.

```
import 'package:mud/box.dart';
import 'package:mud/thing.dart';
import 'package:test/test.dart';

void main() {
  test('on peut créer une boîte', () {
    final b = Box();
    expect(b, isA<Box>());
  });

  test('on peut mettre des choses dedans', () {
    final b = Box();
    b.add(Thing('truc1'));
    b.add(Thing('truc2'));
  });
}
```

```
}
```

Le test échoue, puisque Thing n'existe pas. On commite le test, puis on crée lib/thing.dart :

```
class Thing {  
  String name;  
  
  Thing(this.name);  
}
```

À retenir

Thing(this.name); est un constructeur complet. Il remplace les 4 lignes du constructeur Java :

```
public Thing(String name) {  
    this.name = name;  
}
```



On écrit this.name **dans les paramètres** : Dart range lui-même l'argument dans l'attribut name.

On complète ensuite lib/box.dart :

```
import 'thing.dart';  
  
class Box {  
  final List<Thing> _contents = [];  
  
  void add(Thing thing) {  
    _contents.add(thing);  
  }  
}
```

À retenir



- [] crée une liste vide. Pas besoin de new ArrayList<>().
- Le **soulignement** au début de _contents rend l'attribut **privé**. Dart n'a pas de mot-clé private : c'est le _ qui compte.
- final : on ne remplacera jamais la liste par une autre. On peut quand même **ajouter** des éléments dedans.

Astuce



Une méthode qui tient en une seule expression peut s'écrire avec une flèche :

```
void add(Thing thing) => _contents.add(thing);
```

Lancez les tests, commitez, fusionnez.

Backlog Box et Thing : à vous !

Pour chaque élément, suivez la méthodologie : branche, test qui échoue, commit, implémentation, test qui passe, commit, pull request.

Les tests se regroupent avec group pour rendre l'affichage plus lisible :

```
group('Box : ouverture', () {  
  test('une boîte est ouverte par défaut', () { ... });  
  test('close puis open', () { ... });  
});
```

À retenir

Les *matchers* les plus utiles de package : test :



Matcher	Vérifie que la valeur...
<code>equals(x)</code>	est égale à x
<code>isTrue, isFalse</code>	vaut true / false
<code>isNull, isNotNull</code>	est / n'est pas null
<code>same(x)</code>	est le même objet que x
<code>isA<Type>()</code>	est du type Type
<code>contains(x)</code>	(liste ou chaîne) contient x

La boîte et son contenu

1. On veut pouvoir tester si une chose est dans une boîte : `b.contains(t)` renvoie un `bool`. Une `List` Dart a déjà une méthode `contains` : consultez sa documentation sur <https://api.dart.dev/dart-core/List-class.html>.
2. On veut pouvoir retirer une chose d'une boîte : `b.remove(t)`. Une fois retirée, elle ne doit plus être dedans. Si la chose n'était pas dans la boîte, `remove` renvoie `false`, et `true` sinon.

Astuce



Regardez ce que renvoie la méthode `remove` d'une `List`... c'est exactement ce qu'il nous faut.

Au TP4, retirer une chose absente levait une **exception**. Nous verrons les exceptions Dart au prochain TP. Pour l'instant, on se contente de renvoyer `false`.

Ouvrir et fermer

3. Une boîte peut être ouverte ou fermée. Une boîte neuve est ouverte. On veut pouvoir tester `b.isOpen`, et utiliser `b.close()` et `b.open()`.

Attention



On écrit `b.isOpen`, **sans parenthèses** : c'est un **getter**, qui se lit comme un attribut mais qui est calculé par une méthode :

```
bool _open = true;

bool get isOpen => _open;
```



L'attribut `_open` reste privé : de l'extérieur, on peut le **lire** avec `isOpen`, mais pas le **modifier** directement.

4. On veut qu'une boîte permette `b.actionLook()`. Cette méthode renvoie :
 - "la boîte contient: ceci, cela" si la boîte est ouverte et contient deux choses nommées `ceci` et `cela`;
 - "la boîte est fermée" si la boîte est fermée.

Astuce

Deux outils Dart très pratiques :

- l'**interpolation** : "la boîte contient: \$noms" insère la valeur de la variable `noms` dans la chaîne. Pour une expression, on ajoute des accolades : `'${t.name}'` !
- `map` puis `join` : `_contents.map((t) => t.name)` donne les noms des choses, et `.join(', ')` les colle avec des virgules.



Le volume des choses

5. Une chose peut prendre de la place. On veut pouvoir écrire `Thing('bidule', volume: 3)`, et aussi `Thing('bidule')` : le volume vaut alors 0.
6. On veut connaître le volume d'une chose : `t.volume`.

À retenir

Dart n'a pas de surcharge. Il est impossible d'écrire deux constructeurs `Thing(...)` avec des paramètres différents, comme en Java au TP4.

À la place, on utilise un **paramètre nommé**, entre accolades, avec une valeur par défaut :

```
Thing(this.name, {this.volume = 0});
```



- à l'appel, on écrit le nom du paramètre : `Thing('épée', volume: 3)`;
- s'il est absent, il prend sa valeur par défaut : `Thing('épée')`.

Le code est plus lisible : `volume: 3` dit ce que représente le 3.

Déclarez l'attribut `volume` comme `final int volume` ; le volume d'une chose ne change pas.

La capacité des boîtes

7. On veut pouvoir donner une capacité à une boîte, `b.capacity = 5`, et la lire, `b.capacity`.
8. Quand on n'a pas donné de capacité à une boîte, sa capacité est `null`. Cela signifie « capacité illimitée ».

À retenir



Au TP4, une capacité de `-1` voulait dire « illimitée ». C'était une **valeur magique** : rien dans le type `int` n'indique ce cas particulier.

En Dart, on dit clairement qu'il peut ne pas y avoir de valeur :

```
int? capacity; // le « ? » autorise null
```



- `int` : toujours un entier, jamais `null`. Le compilateur y veille ;
- `int?` : un entier **ou** `null`.

C'est la **null safety** : le compilateur refuse qu'on utilise un `int?` comme un `int` sans avoir vérifié qu'il n'est pas `null`.

9. On veut savoir s'il reste de la place pour une nouvelle chose : `b.hasRoomFor(t)`. Les choses déjà présentes consomment leur volume de la capacité. Une boîte de capacité `null` a toujours de la place.

Astuce

- Pour faire la somme des volumes, une boucle `for (final t in _contents)` suffit. Vous pouvez en faire un getter `int get usedVolume`.
- Si le compilateur refuse `usedVolume + t.volume <= capacity`, c'est à cause de la null safety : `capacity` peut valoir `null`. Recopiez-le dans une variable locale et testez-la :

```
final cap = capacity;  
if (cap == null) return true;  
return usedVolume + thing.volume <= cap; // ici, cap est un int
```



Après le test `cap == null`, Dart sait que `cap` est un `int`.

10. On veut une méthode `b.actionAdd(t)` qui ajoute une chose **seulement** si la boîte est ouverte **et** s'il y a la place. Elle renvoie `true` si l'ajout a eu lieu, `false` sinon.

Nommer et retrouver les choses

11. Une chose doit s'afficher avec son nom : `print(t)` affiche `bidule`, et `'chose: $t'` donne `"chose: bidule"`.

Astuce

Comme en Java, il faut redéfinir `toString`. En Dart, l'annotation `@override` fonctionne pareil :



```
@override  
String toString() => name;
```

12. On veut pouvoir renommer une chose : `t.name = 'truc'`.

À retenir



Au TP4, il fallait écrire `setName`. En Dart, un attribut public suffit : on le lit et on le modifie directement.

Si un jour il faut contrôler la modification, par exemple refuser un nom vide, on remplace l'attribut par un couple `get/set`. Le code qui utilise `t.name` ne change pas.

13. On veut tester si une chose porte un nom donné : `t.hasName('bidule')`.
14. On veut que `b.find('bidule')` renvoie la chose de ce nom contenue dans la boîte. S'il n'y en a pas, ou si la boîte est fermée, `find` renvoie `null`.

Astuce



- Le type de retour est `Thing?` : une chose, ou `null`.
- Pour tester que deux variables désignent **le même objet**, utilisez le `matcher same(t)`.
- Pour chercher, une boucle `for` avec un `return` fait très bien l'affaire. Plus court : `_contents.where((t) => t.hasName(name)).firstOrNull`.

Constructeur de Box

15. On veut pouvoir préciser, à la création, si une boîte est ouverte ou fermée, et sa capacité. Les deux sont facultatifs :

```
Box() // ouverte, capacité illimitée
Box(capacity: 5) // ouverte, capacité 5
Box(open: false) // fermée, capacité illimitée
Box(open: false, capacity: 5)
```

Astuce



Un paramètre nommé peut initialiser un attribut privé grâce à une **liste d'initialisation** (après les `:`) :

```
Box({bool open = true, this.capacity}) : _open = open;
```

Pour finir : un petit programme

Modifiez `bin/mud.dart` pour qu'il crée un coffre de capacité 5, y range une épée (volume 3) et une potion (volume 1), affiche son contenu, puis essaie d'y mettre un bouclier (volume 4) :

```
$ dart run
Bienvenue dans le MUD en Dart!
la boîte contient: épée, potion
Le bouclier ne rentre pas dans le coffre.
```

Question 4

Lancez `dart analyze`. Corrigez tous les avertissements affichés, puis commitez.

Bilan : de Java à Dart

À retenir

Java (TP4)	Dart (TP5)
<pre>new Box() private List<Thing> contents new ArrayList<>() constructeur qui recopie les paramètres plusieurs constructeurs (surcharge) getName() / setName() isOpen() capacité -1 = illimitée "contient: " + noms @Test + assertEquals(attendu, obtenu)</pre>	<pre>Box() final List<Thing> _contents [] Thing(this.name) paramètres nommés {this.volume = 0} attribut public, ou get / set getter isOpen int? et null 'contient: \$noms' test(...) + expect(obtenu, matcher)</pre>



Pour aller plus loin

Si vous avez terminé :

- une boîte vide ouverte devrait afficher "la boîte est vide" avec `actionLook` ;
- ajoutez un getter `remainingRoom` qui donne la place restante (null si la capacité est illimitée) ;
- deux choses de même nom et de même volume devraient être considérées comme égales : re-définissez l'opérateur `==` et `hashCode` dans `Thing`. Documentation : <https://dart.dev/effective-dart/design#equality>.

Rappel

Au prochain TP, on verra :

- les **exceptions** en Dart, pour remplacer les `false` et `null` de ce TP quand ils signalent une erreur ;
- le **JSON**, pour sauvegarder et recharger le contenu des boîtes dans un fichier.

